How Fenxi Proofer Actually Works

A plain-English walkthrough of the codebase - what it does, how each piece fits, what it can't do yet, and where it should go next.

This is a tool that compares two packaging files - the **approved artwork** and a **sample** (a reprint, a new batch, a supplier's proof) - and tells you every place they differ, how serious each difference is, and why it fired. Everything runs on your own machine; nothing is uploaded anywhere.

> *Think of it as a very fast, very literal proofreader with a ruler, a barcode scanner, a colour meter and a spell-checker built in - one that never gets tired on page 47 of a leaflet and never "corrects" a wrong number because it looked like a typo.*

1. The shape of the project

Everything lives in one folder. Here is what each piece is actually for:

File	What it's the brain for
`app.py`	The web server (FastAPI). Handles uploads, runs jobs in the background, serves the report UI.
`proofer/compare.py`	The engine. ~1,800 lines - opens both PDFs, runs every check, merges the results into one list of findings.
`proofer/align.py`	Figures out if the sample is just shifted / rotated / scaled / photographed, and straightens it before anything else runs.
`proofer/copydeck.py`	Checks the artwork against an approved Word/text document, not just against the old PDF.
`proofer/spelling.py`	Dictionary-based spell check, multiple languages plus your own word list.
`proofer/ocr.py`	Wraps RapidOCR - reads text out of pixels when there's no real text to read.
`proofer/imaging.py`	New: turns a JPG/PNG into a real one-page PDF so the rest of the pipeline can treat it normally.
`proofer/profiles.py`	Tolerance presets (Standard / Strict / Lenient / Print-run) and the "this field is allowed to change" rules.
`proofer/jobs.py`	Runs a proof as a background job, stores results on disk, keeps the tamper-evident review/sign-off log.

File	What it's the brain for
`proofer/pairing.py / batch.py`	Batch mode: matches up reference+sample pairs in a folder by filename, runs them all.
`proofer/annotate.py`	Writes every finding onto the sample PDF itself as a real, clickable PDF comment.
`proofer/report.py`	Turns a finished proof into JSON / CSV / a standalone offline HTML report.
`proofer/web/`	The browser UI - plain HTML/CSS/JS, no build step, no framework.

2. What happens when you run one proof

Whether you use the web app, the command line, or batch mode, the same seven steps happen underneath. This is the part worth understanding cold, because it explains almost every design decision in the codebase.

1. **Upload / point at two files**
   You give it a **reference** (the approved file) and a **sample** (the one to check), plus optionally an approved-text document (the "copy deck"). The web app checks file size and file type before anything else runs.

2. **Make sure both sides are real PDFs**
   If a file is a JPG or PNG, it gets wrapped into a genuine one-page PDF right here (`proofer/imaging.py`). This matters more than it sounds - see section 6.

3. **Pair up the pages**
   Page count differs? Pages swapped? The engine matches pages by their actual content (not just position 1-to-1, 2-to-2), so an inserted or removed page is reported once instead of quietly shifting every later page comparison out of sync.

4. **Straighten the sample ("registration")**
   Before comparing anything else, the engine checks: is this just the same artwork, shifted / scaled / rotated / photographed at an angle? If so, it works out that one transform, reports it once ("whole page scaled to 98.5%"), and aligns the sample into the reference's coordinate frame. Skip this step and a slightly crooked scan produces fifty fake "moved" findings that bury the one real change.

5. **Run every check, independently, per page**
   Text, barcodes, spelling, colour, layers, structure, Braille, ink plates, spot colours, fine print, OCR - each check runs on its own and has no idea what the others found. This is deliberate: independent checks can't share the same blind spot.

6. **Merge everything into one finding per real difference**
   If three checks all react to the same physical spot (say, a changed number that also breaks a barcode's printed caption), the engine merges them into a single finding instead of reporting the same problem three times.

7. **You review, and sign off**
   The web UI shows both pages side by side (or overlaid, or a swipe/flicker view). You accept or reject each finding, then sign off. Every decision - who, when, what - is written to an append-only log where each entry is hash-chained to the one before it, so editing history breaks the chain and the report says so.

Exit codes if you're scripting this: **0** = PASS, **1** = FAIL (a major/critical difference), **3** = REVIEW (only minor differences, or a human should look), **2** = usage error.

3. The checks, one at a time

Each of these is a separate function in `compare.py` (or its own module). None of them trust each other - each one looks at the raw file and forms its own opinion.

Text

Do the words say the same thing?

Words are read straight out of the PDF's text layer - not a picture of them, the actual text objects. Words are grouped into paragraphs by their position on the page, matched between the two files, then diffed word by word. Digits and units (mg, ml, %) are treated as critical: a changed dose is never "just a wording difference".

Copy deck (approved text)

Does the artwork match the approved Word document, not just the old PDF?

Each approved paragraph is located on the sample wherever it actually sits - panel order and line breaks don't matter - then diffed word by word. Also flags approved text that's missing entirely, and artwork text that no approved paragraph accounts for.

Spelling

Any words that shouldn't be there?

Seven language dictionaries plus your own `dictionary.txt` (brand names, INNs). Only genuinely new unknown words become findings - if the approved reference already contains the word, it's not flagged again.

Barcodes

Will this actually scan at the till or in the warehouse?

Every symbology the zxing-cpp library can read (EAN, Code128, DataMatrix, QR...). Checks: does the payload match, does the check digit still work, is it rotated or moved or resized, does the printed human-readable number under the bars agree with what the bars actually encode, and a print-quality proxy (contrast, reflectance, quiet zone) that predicts whether it'll scan at all.

Braille

Does the Braille say the right thing?

The dots are read directly from the PDF's vector drawing commands (the little raised-dot circles) and decoded to letters, grade 1 English, checked both sides.

Colour

Did a colour shift?

Fill and stroke colours are read straight from the PDF's own paint commands, plus a CIE-Lab comparison of the rendered pages (CIE-Lab is a colour space built to match how far apart two colours *look* to a human eye, not just their raw RGB numbers).

Layers

Did a varnish or spot-finish layer get toggled?

PDFs can have optional-content groups (OCGs) - layers that can be turned on/off, often used for varnish or foil. This check catches one being added, removed, or silently toggled.

Ink separations (CMYK plates)

Is there a change that's invisible on screen but will show up on a printing press?

Both pages are rendered to CMYK and each ink plate (Cyan, Magenta, Yellow, Black) is compared on its own - including K-only black rebuilt as "rich black" and colours rebuilt from completely different ink combinations that happen to look the same on a monitor.

Spot colours & overprint

Did a named Pantone ink get swapped, or did overprint get switched on/off?

Read directly from the PDF's own colour-space objects, not inferred from pixels.

Page structure

Page count, size, rotation, fonts, image resolution?

Catches an inserted/removed page (without it shifting every later page's comparison), a trim/bleed box change, an un-embedded font, or a sample image whose effective resolution dropped.

Visual

Everything the checks above don't have words or numbers for.

Both pages rendered at 300 dpi, compared in CIE-Lab, changed pixels clustered into regions, each region classified as moved / removed / added / recoloured / restyled. A "move" has to re-align almost perfectly after sliding it - otherwise it's an edit wearing a disguise, not a move.

Fine print on low-res images

Did a tiny number change on a scanned/flattened pack?

The suspicious region is OCR'd in the sample *and* in a version of the reference deliberately degraded to the same scan quality. A change is only reported if the degraded reference still reads correctly - so a plain OCR misread can't raise a false alarm.

OCR (image-only artwork)

What if there's no real text layer at all?

RapidOCR (a dedicated text-reading model, not a language model - see section 5) reads the changed regions and barcode captions, and the copy-deck check runs on that OCR'd text instead of a text layer.

4. The trick that makes this actually usable: registration

This is the single cleverest thing in the codebase, and the reason the tool doesn't drown you in false alarms.

Before any content check runs, `align.py` asks one question: "is the sample just the same artwork, moved?" It tries two independent ways to answer that:

- **Text-based:** find words that appear exactly once on each page, match them by their text, and see if their positions imply one consistent shift/rotate/scale.
- **Feature-based (for image-only artwork or photos):** detect visual keypoints (ORB features) on both renders, match them, and fit a transform using RANSAC - a method that ignores outliers, so a handful of genuinely different pixels can't fool it into reporting a fake "whole page moved".

If a transform is found and most of the page supports it, the whole thing gets reported **once** ("whole page scaled to 98.5%, rotated +0.3°") and the sample is re-rendered straight into the reference's coordinate frame before anything else runs. A phone photo taken at a slight angle, or a re-imposed file shifted by a millimetre, is handled the same way - and if it's a genuine photo, the report also flattens out the photo's lighting gradient and marks that page as "limited coverage", so a clean-looking result on a rough photo is never quietly passed off as a full check.

5. Turning ten checks' worth of noise into one trustworthy answer

One real difference, one finding

If the text check, the barcode check and the visual check all react to the same spot, they get merged. Nobody wants to see "barcode payload changed", "visual content differs" and "text changed" as three separate line items for one typo in a lot number.

Severity: info / minor / major / critical

PASS = nothing found. **REVIEW** = only minor differences, or something needs a human eye (e.g. a page had no text layer). **FAIL** = a major or critical difference.

Confidence: high vs medium

High means the finding was read directly from the file's own data (real text, a decoded barcode payload, a PDF colour object). **Medium** means it was inferred from pixels or OCR - still worth flagging, but the report says so, and a reviewer should weigh it accordingly.

OCR is a describer, never a judge

Why not just ask an LLM to read the label?

This is worth being precise about, because it's a real design choice, not an accident: **OCR never decides two files don't match.** It only runs on a region the pixel-diff check already flagged, to put a caption on it ("reads '0.5 mg' in the reference vs '5 mg' in the sample"). And it's a real OCR model (RapidOCR, ONNX-based) rather than a language model, on purpose: a language model would "helpfully" autocorrect a wrong number back to what it expects to see - exactly the mistake this tool exists to catch, not paper over.

6. Making JPG and PNG actually work (not just "accepted")

This was today's addition, and it's a good illustration of how the rest of the pipeline assumes real PDF structure everywhere.

The naive approach breaks

The PDF library (PyMuPDF) will happily *open* a raw JPG or PNG - it treats it as a one-page "document". So the first instinct is: just let those files through untouched. That actually **segfaults** the whole Python process partway through a proof, because deeper checks assume PDF-specific structure that a bare image simply doesn't have - for example, the registration step (section 4) does this to render an isolated copy of a page:

```
iso = pymupdf.open()
iso.insert_pdf(page.parent, from_page=page.number, to_page=page.number)
```

`insert_pdf` expects to copy real PDF page objects from one PDF into another. Feed it a page that's secretly just a raw image with no page tree, no /Resources dictionary, nothing - and the C library underneath crashes outright instead of raising a catchable Python error.

The fix: wrap the image into a real PDF, once, up front

`proofer/imaging.py` sniffs the uploaded bytes (by content, not filename) and, if it's a JPG or PNG, converts it into a genuine single-page PDF - the image placed as a normal image object on a normal page - before it ever reaches a single check:

```
def open_pdf(path):
  data = Path(path).read_bytes()
  if not is_pdf(data) and is_image(data):
    data = to_pdf_bytes(data) # PyMuPDF's own convert_to_pdf()
  return pymupdf.open(stream=data, filetype="pdf")
```

Every place that opens a file - the comparison engine, the annotated-PDF export, the CLI, batch mode - now goes through this one function. The web app also converts at upload time, so whatever's saved to disk for a job is always a real PDF.

The result: a JPG/PNG now runs through exactly the same code path a flattened, image-only scanned PDF already used. No text layer to diff word-by-word, so text and copy-deck checks fall back to visual comparison plus OCR - but barcode, colour, layout and registration checks work identically either way.

How we know it actually works, not just that it doesn't crash

Verified with real jobs through the actual running app, not just by reading the code: PNG-vs-PNG, PDF-vs-JPG, and a real photographed/JPEG-compressed sample all proofed correctly end to end (including the annotated-PDF export, which is the step that used to be most at risk of hitting the same crash). A clean JPG snapshot of unchanged artwork correctly came back with zero false defects.

7. Profiles: how picky should this proof be?

A "Config" object holds every tolerance the checks use (colour difference threshold, how far something can move before it counts, barcode rotation tolerance, etc). A profile is just a named bundle of overrides to that config:

Profile	What changes
Standard	The validated defaults. Use this unless you have a reason not to.
Strict	Smaller colour / position / size changes count - for regulated artwork approval.
Lenient	Quick screening; minor differences are noted as information, not flagged as findings.
Print run check	Lot / batch / expiry / serial numbers are *expected* to differ. A change right after one of those labels is reported as "variable data, as expected" instead of a defect - but only under this profile. Run the same file through Standard and a changed expiry date is still critical.

Individual numbers (colour tolerance, position tolerance, minimum mark size, barcode shift, dpi) can also be overridden directly, and every proof records the exact settings it ran with, so a result is always reproducible later.

8. What comes out the other end

- **Interactive report** - side by side, overlay, swipe, and flicker views; accept/reject each finding; sign off with a hash-chained audit log.
- **Annotated PDF** - the sample file itself with every finding drawn on as a real PDF comment (opens in Acrobat), plus a summary page in front.
- **Standalone HTML** - a fully offline report, images embedded, nothing to fetch.
- **CSV** of findings, **JSON** of the full proof.
- **Batch mode** adds a ZIP of every report plus one combined CSV across the whole folder.

9. Current limitations - stated plainly

These are the honest boundaries of what exists today, some from the project's own README, some we confirmed directly while adding JPG/PNG support.

About accuracy

- The tool was tuned on synthetic sample designs. The measured 82/82 and 540/540 detection numbers are real, but they're measured against those designs - a claim about a specific client's real artwork needs that client's own approved PDFs run through `mutate.py`.
- Mutation testing only measures the defect *types* it knows how to inject. It doesn't replay a team's actual historical printing errors unless those are added too.
- On resampled image-only artwork (e.g. a 150 dpi scan), text under 6pt can be missed by the pixel check - the report warns when a page is thin on resolution like this.

About what's technically measured

- Colour is vector colour values plus rendered RGB, not a true CMYK/spot-separation instrument reading.
- Barcode grading is a print-quality *proxy* (contrast, reflectance, quiet zone) - not an ISO/IEC 15416 certified grade.
- Braille is grade 1, English only.
- Not validated for regulated (GxP / 21 CFR Part 11) use - the audit trail is tamper-evident, but formal validation is separate, unstarted work.

About the JPG/PNG support specifically

- An image has no live text, so word-for-word text and copy-deck checks fall back entirely to OCR - meaningfully weaker than a live-text PDF. Use a PDF on at least one side whenever you can.
- JPEG compression adds pixel noise that can trigger extra visual findings that aren't real defects - we saw this directly in testing (three extra minor/major findings on an otherwise-identical JPEG render). This isn't a bug in the new feature; it's the same caveat that already applies to any lossy image-only artwork.
- The conversion step assumes a single image = a single page. A multi-page scenario needs one image per page today; there's no batch "12 photos = 1 document" assembly yet.

About the engine generally

- Overprint is *detected* (a flag read from the PDF) but not *simulated* - it doesn't render what overprinting two inks would actually look like.
- It's a single local process - fine for one proofer at a desk, not built for concurrent multi-user load, roles, or permissions.

- No malformed/adversarial-PDF hardening was reviewed as part of this session; a hostile input file wasn't in scope of what we tested today.

10. Where this could go next

From the project's own roadmap

1. Tune the thresholds on a specific client's own historical artwork and past errors, not just the synthetic sample set.
2. Simulate overprint (render what two overlapping inks actually look like), and render spot-colour plates properly.
3. Real ISO/IEC 15416 / 15415 barcode grading instead of the current proxy.
4. Grade 2 and non-English Braille.
5. An optional AI "second reader" on image-only artwork - explicitly additive (it can only add findings, never remove ones the deterministic checks already found), off by default.
6. Multi-user support: accounts, roles, and a formal validation package for regulated use.

Natural extensions of the JPG/PNG work

1. Multi-photo assembly: let someone drop 3-4 photos of one folded leaflet and stitch them into one multi-page document, instead of requiring one image per page.
2. Automatic perspective correction quality check: warn explicitly when a photo's registration confidence is too low to trust the rest of the proof, rather than just proceeding with "limited coverage".
3. A pre-flight sharpness/exposure check on photo uploads, so a blurry phone photo is rejected with a clear message before it burns two minutes of processing.

Smaller, practical wins

1. Surface the exact tolerance profile and any overrides directly on the report header, not just in the settings screen - useful when someone else re-opens a report cold.
2. A "diff of diffs": when re-running a proof after a supplier fix, show only what changed since the last run instead of a full fresh finding list.
3. Expose `PROOFER_WORKERS` and batch concurrency in the UI so a laptop with fewer cores doesn't get set up to thrash.

Generated as a plain-language map of the codebase as it stands, including the JPG/PNG support added in this session. Everything above was checked against the actual source, not summarized from memory.